

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

College of Computing and Data Science

Nanyang Technological University

SC3020 Database System Principles Project 2

AY 25/26 Semester 2 | Group 20

Name	Email	Matriculation Number
Caleb Yeo	CYEO039@e.ntu.edu.sg	U2323500G
Gay Ming Kai	C230114@e.ntu.edu.sg	U2320596C
Lim Zu Liang	E230184@e.ntu.edu.sg	U2322044H
Lin Hongchao	LINH0039@e.ntu.edu.sg	U2322957G
Wang Siwen	WANG2054@e.ntu.edu.sg	U2322296G

Contribution Matrix

Name	Contribution (%)
Caleb Yeo	20%
Gay Ming Kai	20%
Lim Zu Liang	20%
Lin Hongchao	20%
Wang Siwen	20%

1. Introduction **4**
2. System Overview **4**
3. Algorithms & Annotations **5**
3.1. Overview 5
3.2. QEP Node Traversal 6
3.3. Scan Annotation Logic 7
3.4. Join Annotation Logic & AQP Cost Ratios 8
3.5. SQL Clause Mapping 9
4. GUI **10**
4.1. Application Layout 10
4.2. Top Bar & Database Connection 10
4.3. Left Panel: SQL Query Input 11
4.4. Middle Panel: Annotated Output 11
4.5. Right Panel: QEP Visualisation 14
4.5.1. QEP Tree 15
4.5.2. Cost Chart 16
4.6. History Logs 17
4.7. Startup Checks 17
5. Query **19**
5.1 Sequential Scan 19
5.2 Index Scan 23
5.2.1 Index-Only Scan 25
5.3 Nested Loop 27
5.4 Hash Join 28
5.5 Merge Join 30
5.6 More Complex Query Examples 31
5.7 Handled Exceptions 33
6. Limitations **35**

1. Introduction

This project aims to develop an application that can explain SQL queries with simple English explanations for what each clause does, as well as the backend plans generated by the database for execution of said query. This is accompanied by a visual representation of the Query Execution Tree showing the flow of data from access to the final query result. In doing so, we aim to make database implementations and queries more accessible to people with limited understanding of RDBMS functionality. Within organisations, people with varying levels of expertise need to interact with database systems, from finance to developers. Additionally, contemporary AI-assisted programming has led to increased accessibility to less experienced users to implement their own solutions. Simply accepting the RDBMS as a black box can lead to confusion and misunderstanding how the query results ought to be interpreted. Therefore, this project bridges the gap by using accessible language to improve the explainability of how the RDBMS functions, which improves user efficiency.

2. System Overview

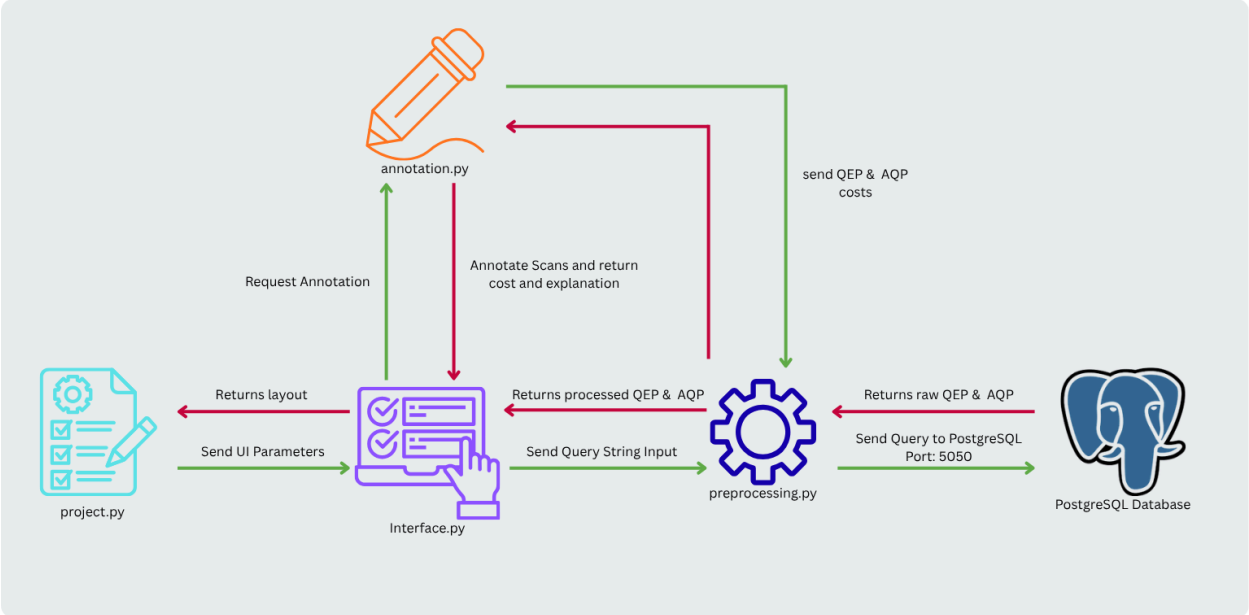


Figure 1. System Design Diagram

The system is built on 4 python modules (***project.py*** , ***interface.py***, ***preprocessing.py*** and ***annotation.py***). ***Project.py*** acts as the entry point, configuring the Tkinter environment and launching the GUI application. ***Interface.py*** creates the full 3-panel GUI and orchestrates the pipeline. When it receives an user query input, it forwards it to ***preprocessing.py*** to connect to the PostgreSQL database (Port 5050) to retrieve the raw QEP and AQP data. It then processes it into a structured format and returns it to ***interface.py***. The data is then passed to ***annotation.py*** which analyses the query components alongside the cost to produce the annotation and returns it back to ***interface.py*** for display.

Detailed descriptions of the annotation algorithm and GUI design are covered in Sections 3 & 4 respectively.

3. Algorithms & Annotations

3.1. Overview

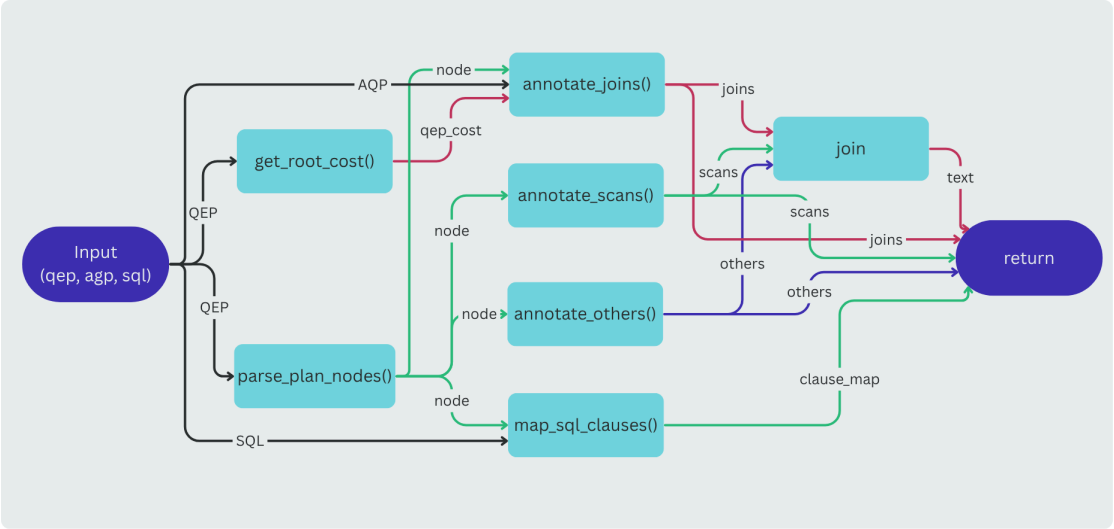


Figure 2. generate_annotations() flowchart

The annotation pipeline is implemented in annotation.py and is invoked by the GUI layer after preprocessing.py retrieves the required plan data. It receives three inputs: the selected Query Execution Plan (QEP), a dictionary of Alternative Query Plans (AQPs), and the original SQL query text.

The master entry point is generate_annotations(). It first flattens the nested QEP tree into a linear list of operator nodes using parse_plan_nodes(). The flattened nodes are then passed into separate annotation components for scan operations, join operations, and other physical operators such as Sort, Aggregate, Limit, Gather, and Unique.

Besides generating grouped explanations, the pipeline also produces a clause map through map_sql_clauses(). This clause map aligns SQL lines with the most relevant execution notes for inline display in the middle panel. Since PostgreSQL may reorder joins, push down filters, or transform subqueries during optimization, this mapping is heuristic rather than a guaranteed one-to-one mapping.

Finally, generate_annotations() returns a structured dictionary containing scan annotations, join annotations, other-operator annotations, the clause map, and a concatenated full_text string used by the interface layer.

3.2. QEP Node Traversal

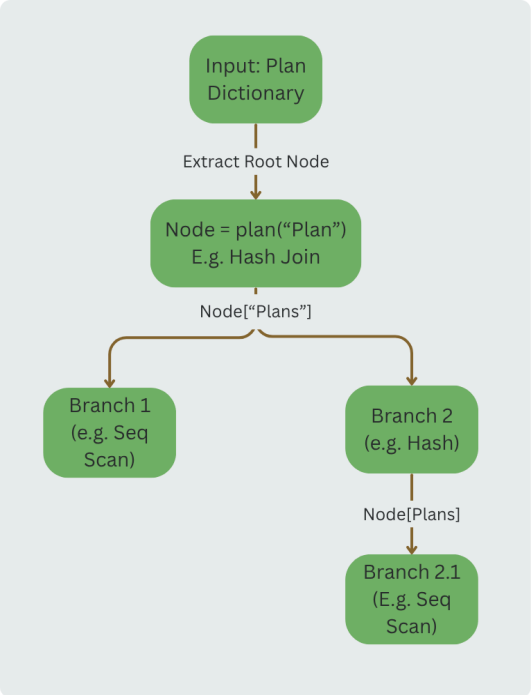


Figure 3. QEP Tree Data Structure

parse_plan_nodes() is a recursive function that traverses the nested QEP plan tree and returns a flat pre-order list of operator nodes. It first extracts the root plan node from the "Plan" field when present, appends it to an accumulator list, and then recursively processes each child node listed under the "Plans" array.

This flattened node list is then passed to the annotation routines, which filter nodes by their Node Type and classify them into scan nodes, join nodes, and other physical operator nodes.

Groups	**Operations**
Scan Nodes	Seq Scan, Index Scan. Index Only Scan, Bitmap Heap Scan. Bitmap Index Scan, Tid Scan
Join Nodes	Hash Join, Merge Join, Nested Loop
Other Nodes	Sort, Aggregate, Hash Aggregate, Group Aggregate, Limit, Unique, Materialize, Gather, Gather Merge, Subquery Scan, BitmapAnd, BitmapOr

3.3. Scan Annotation Logic

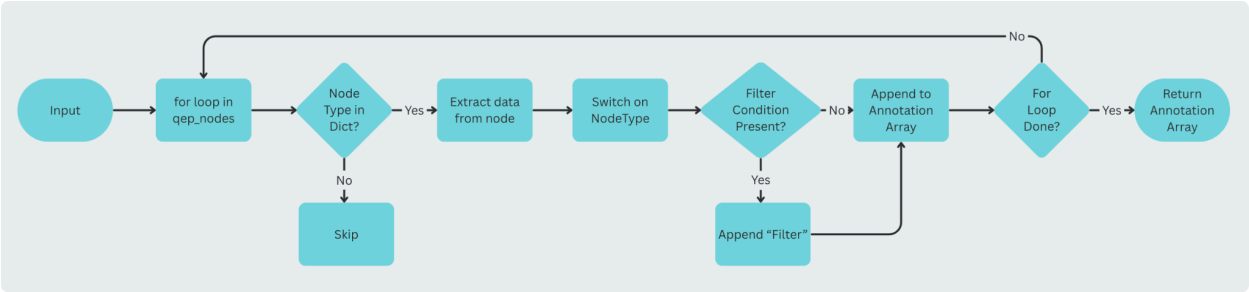


Figure 4. Scan Annotation Pipeline

annotate_scans() iterates over all QEP nodes and processes only those whose Node Type is classified as a scan operator. For each scan node, it extracts key fields such as:

- Relation Name (table)
- Alias, if present
- Index Name, if present
- Filter, Index Cond, and Recheck Cond, if present
- Rows Removed by Filter, if present

It then generates a plain English explanation based on the scan type:

- Sequential Scan: explains that PostgreSQL estimated the sequential scan to be the cheapest access strategy, which may occur due to small table size, low selectivity, or the absence of a useful index.
- Index Scan: explains that an index is used to locate matching rows directly, avoiding a full table scan.
- Index Only Scan: explains that the required columns can be satisfied from the index, reducing or avoiding heap access.
- Bitmap scans: explain the bitmap construction and subsequent heap fetch strategy.
- TID Scan: explains direct tuple id access and its limited applicability.

If additional conditions are present, such as Filter, Index Cond, Recheck Cond, or Rows Removed by Filter, they are appended to the annotation to provide more execution context.

3.4. Join Annotation Logic & AQP Cost Ratios

annotate_joins generates join annotations for join operators present in the QEP (Hash Join, Merge Join, Nested Loop). For each join node, it extracts:

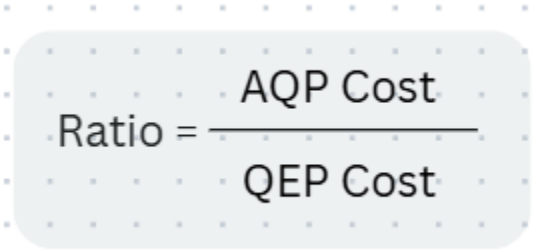
- Join Type (e.g., Inner, Left), when available
- Join condition, chosen by operator-specific fields:
    - Hash Join: Hash Cond
    - Merge Join: Merge Cond
    - Nested Loop: Join Filter or Filter
- Join Filter, if present and distinct from the chosen primary condition
- The node's Total Cost (when available), reported as a per-node cost detail

The primary join annotation describes the algorithm used:

- Hash Join: build a hash table on one input and probe it with the other.
- Merge Join: sort both inputs on the join key and merge them.
- Nested Loop: for each outer row, scan/probe the inner input for matches.

AQP cost ratios are used only to produce an optional whole-query comparison summary. The implementation compares the QEP root cost to the root cost of specific AQPs where a join operator is disabled (e.g., "no_hashjoin", "no_mergejoin", "no_nestloop"), and reports cases where the alternative plan is more expensive.

The cost ratio is computed by compare_costs() as:



For each relevant AQP label, PostgreSQL replans the entire query with a specific join operator disabled using SET LOCAL <operator> = off. The alternative plan's root cost is retrieved using get_root_cost(). If the ratio is greater than 1.0, the alternative whole-query plan is considered more expensive than the selected QEP.

In the final annotation output, AQP cost comparison is displayed as a separate summary rather than repeated under every join node. This avoids confusing the whole-query comparison with the per-node cost of an individual join operator. Each join node reports its own Total Cost, while the AQP summary compares the original QEP root cost against alternative whole-query root costs.

3.5. SQL Clause Mapping

map_sql_clauses() generates a line by line mapping from the SQL query text to concise execution notes suitable for inline display. The function:

1. Splits the SQL text into lines.
2. Builds lookup structures from the QEP node list:
    a. scan_notes: maps table names and aliases (lowercased) to short scan descriptions derived from scan nodes.
    b. join_notes: stores join operator types with their most relevant join condition text (Hash Cond / Merge Cond / Join Filter / Filter).
    c. other_map: stores short notes for ORDER BY (Sort), GROUP BY/HAVING (Aggregate variants), LIMIT, and DISTINCT (Unique) if present in the plan.
3. Iterates through SQL lines and assigns annotations based on leading keywords and simple “in FROM clause” state.

Mapping behavior by clause type:

- FROM and FROM-continuation lines: annotate by matching table/alias tokens against scan_notes.
- JOIN lines: combine any matched scan notes with a join note. The function attempts to match the SQL ON condition to a QEP join condition by extracting column tokens and comparing them; if no suitable match is found, it falls back to consuming the next unused join note sequentially.
- WHERE and standalone ON lines: first attempt scan-note matching; if no scan match is found, it attempts join-note matching (condition-based first, sequential fallback second).
- ORDER BY, GROUP BY, HAVING, LIMIT, DISTINCT: mapped using other_map when the corresponding operators exist in the QEP.

The returned clause_map is a list of (sql_line, annotation) pairs. When multiple notes apply, they are combined into a single annotation string separated by “ | ”, which the GUI later renders as multiple inline bullet items under the corresponding SQL line.

4. GUI

4.1. Application Layout

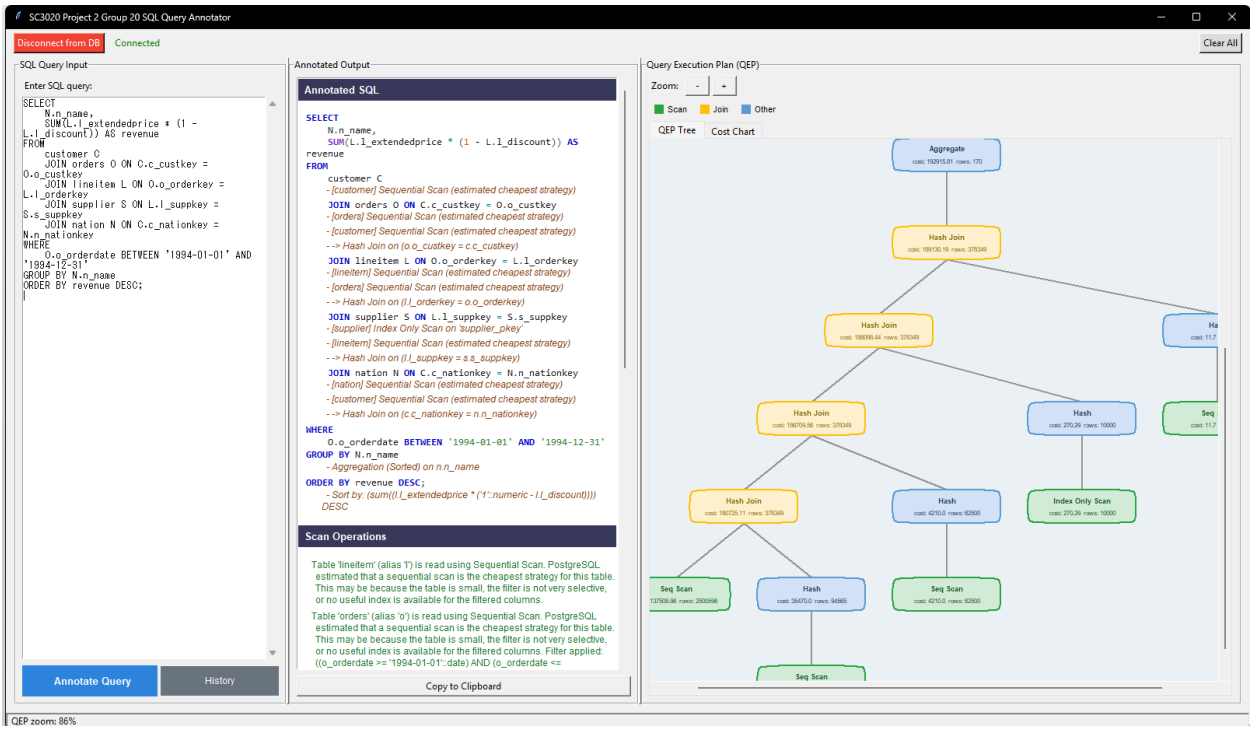


Figure 5. Application layout

The GUI is built using Python's tkinter library and is launched via ***project.py,*** which handles the pre-launch checks before instantiating the **TpchAnnotatorApp** class defined in ***interface.py.*** The main window is structured as a horizontal paned window with 3 resizable panels. SQL query input window, annotated output window and the QEP window. A persistent top bar sits above all the 3 panels and a status bar runs along the bottom of the window to display status messages such as "Fetching QEP" or "Done".

4.2. Top Bar & Database Connection



Figure 6. Connection Interface

The top control bar contains a ***Connect to DB*** button that calls get_connection() from preprocessing.py to establish a psycopg connection using credentials from the .env file which toggles to ***Disconnect from DB*** that calls close_connection() from the same file to disconnect from psycopg safely. A connection status label immediately to its right updates to Connected on success and Not Connected on failure.

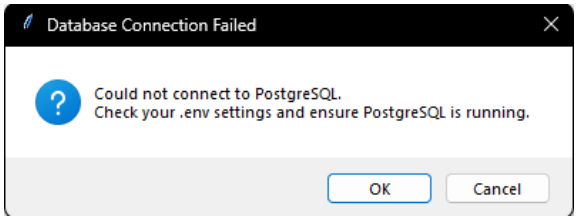


Figure 7. Failed Connection Messagebox

During failure, a dialogue box will prompt the user to check their .env settings and verify that PostgreSQL is running.



Figure 8. Clear All Button

A ***Clear All*** button on the right side of the top bar resets the query input, annotated output panel and QEP canvas.

4.3. Left Panel: SQL Query Input

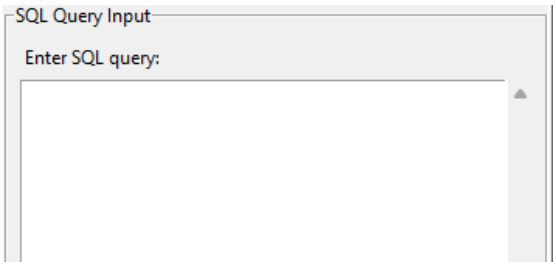


Figure 9. SQL Query Input

The left panel provides a ***ScrolledText*** widget for the user to type or paste a SQL query.

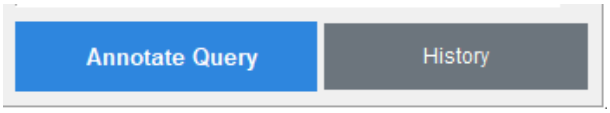


Figure 10. Bottom Buttons Layout

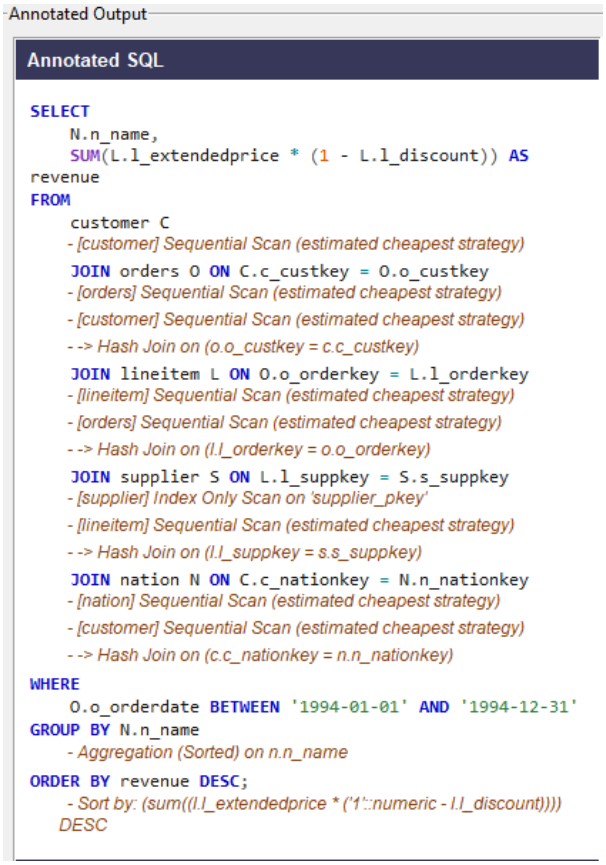
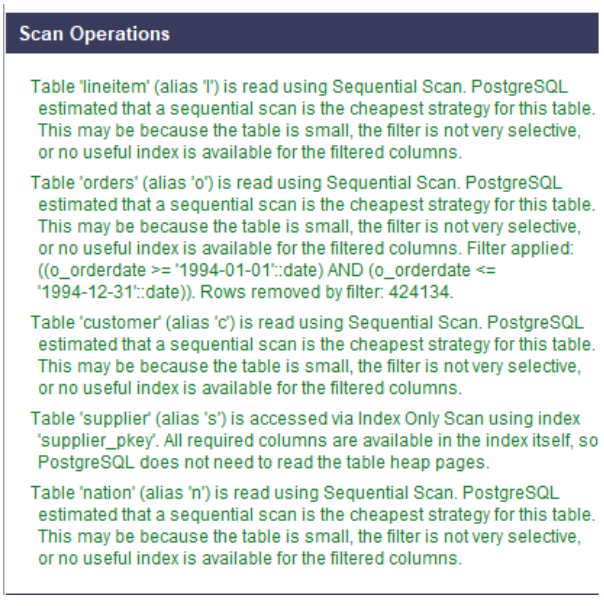
2 buttons sit below the text box in a side-by-side layout.

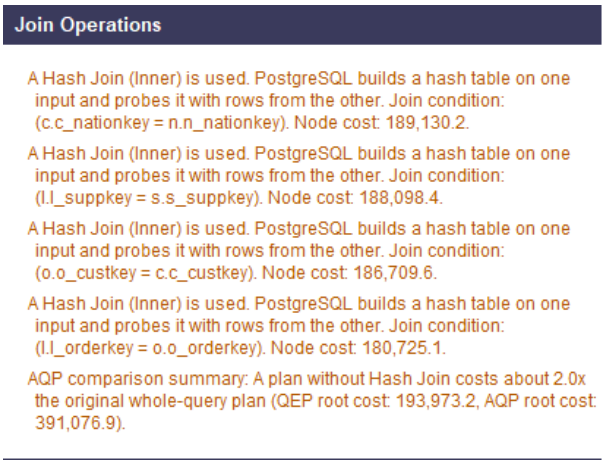
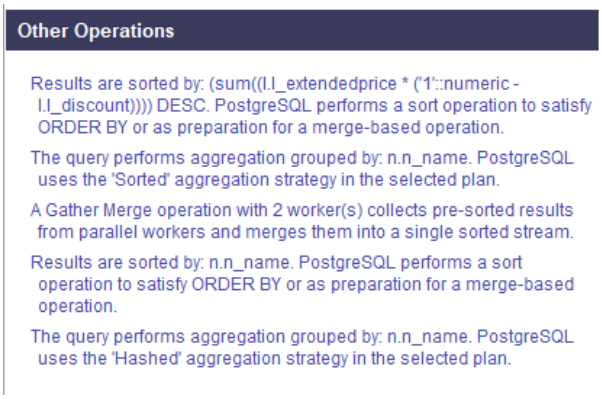
- Annotate Query: Triggers the full annotation pipeline via ***run_annotation()***
- History: Opens the annotation history popup window.

When annotate query is clicked, the run_annotation function would first checks if a database connection is active, then it checks the query using the validate query function from preprocessing.py to reject any invalid queries from empty inputs, multi-statement queries, non-SELECT statements and dangerous keywords. If validation passes, a modal window would show a progress bar which the pipeline executes.

4.4. Middle Panel: Annotated Output

The middle panel is divided into 4 color coded sections to display the results from annotation.py.

Section	Content	Snippet
Annotated SQL	Each SQL line with annotated execution notes indicated below	
Scan Operations	1 entry per scan node	

Join Operations	1 entry per join node with AQP cost ratios	
Other Operations	For other miscellaneous operations such as Sort, Aggregate and Limit	

A copy to Clipboard button at the bottom of the panel copies the full annotated output as plain text.

4.5. Right Panel: QEP Visualisation

The right panel displays an interactive query execution plan for users to understand how PostgreSQL executes the submitted SQL query. This panel is organised as a Tkinter notebook with two tabs, users can toggle between the QEP Tree tab to examine the operator flow or the Cost Chart Tab to view the plan cost.

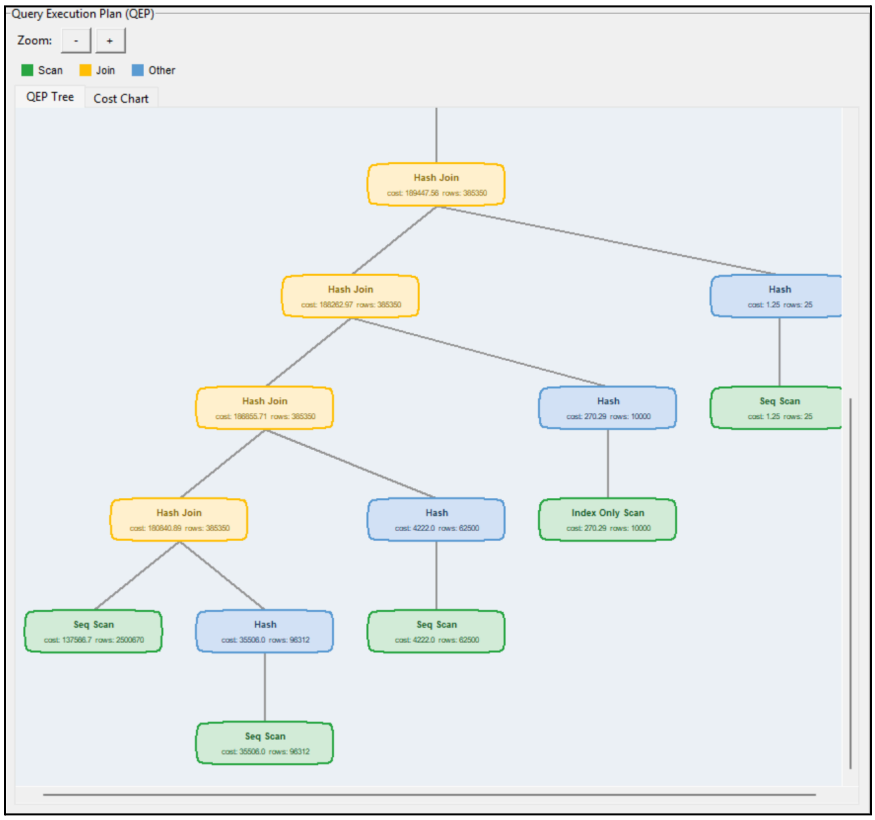


Figure 11. QEP Visualisation Panel: QEP Tree

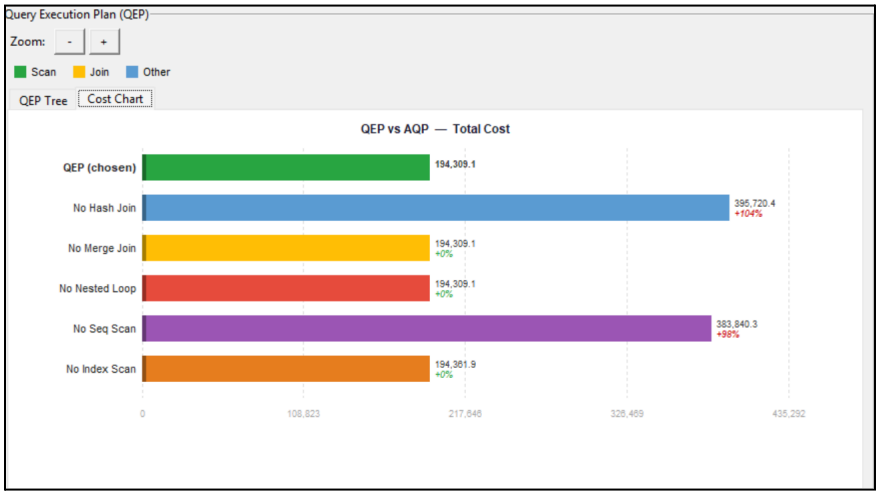


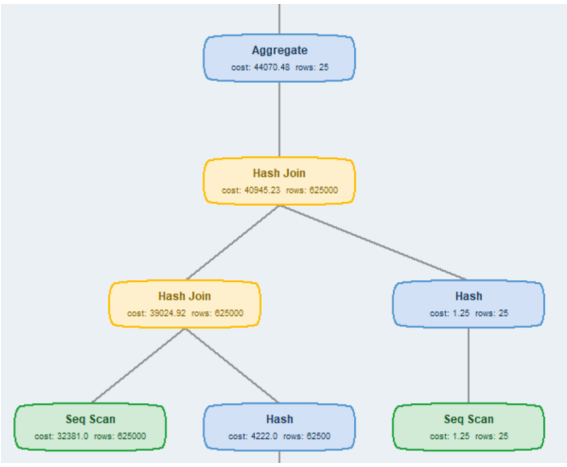
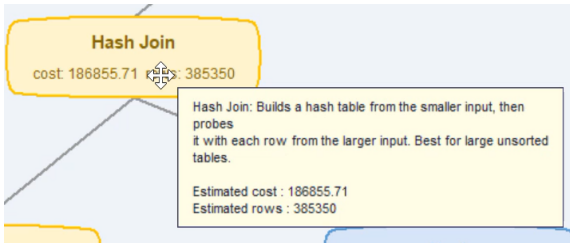
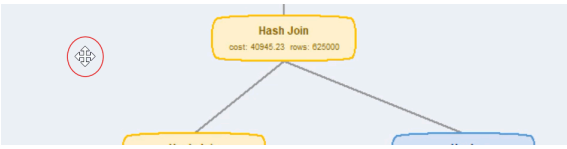
Figure 12. QEP Visualisation Panel: Cost Chart

4.5.1. QEP Tree

The QEP Tree is based on the plan JSON returned by PostgreSQL EXPLAIN output. It allows us to trace how the query was executed from start to finish, with each tree node representing a physical operator.

The tree nodes are color-coded by operator type to distinguish between scans, joins, and other operators at a glance. In the hierarchy of operators, the root node represents the final output operation, while the child nodes are lower-level operations used to produce intermediate results. Hovering over each node provides additional details about the operator, its estimated total cost and estimated number of resulting rows.

To improve the readability of the tree, users are provided with zoom controls at the top of the panel. In addition, the interface also supports drag or scroll gestures to pan across the tree.

Feature	**Description**	**Snippet**
Colour-Coded Nodes	Green: - Scans (e.g., Sequential Scan, Index Scan). Yellow: - Joins (e.g., Hash Join, Merge Join, etc). Blue: - Others (e.g., Sort, Aggregate, Limit).	
Node Tooltips	Hover over to see additional operator details.	
Zoom Controls	Zoom in/out to inspect large or dense plans.	
Pan/Scroll Support	Drag or scroll to pan the tree view.	

4.5.2. Cost Chart

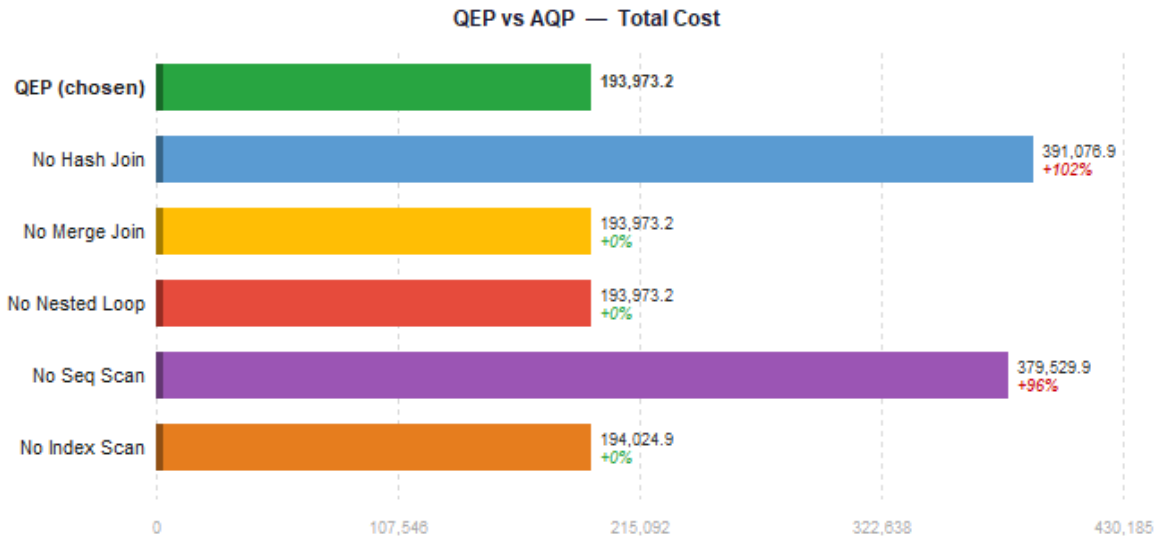


Figure 13. Cost Chart

The cost chart tab tenders a horizontal bar chart on a Tkinter's Canvas. The QEP's total cost is shown as the first green bar, followed by 5 more AQP bars represented by

- No Hash Join AQP
- No Merge Join AQP
- No Nested Loop AQP
- No Seq Scan AQP
- No Index Scan AQP

Each bar displays its cost value as label and the AQP bars also show a percentage label in red/green relative to the QEP cost (In theory it should always be red but due to rounding purposes 0% is considered green).

4.6. History Logs

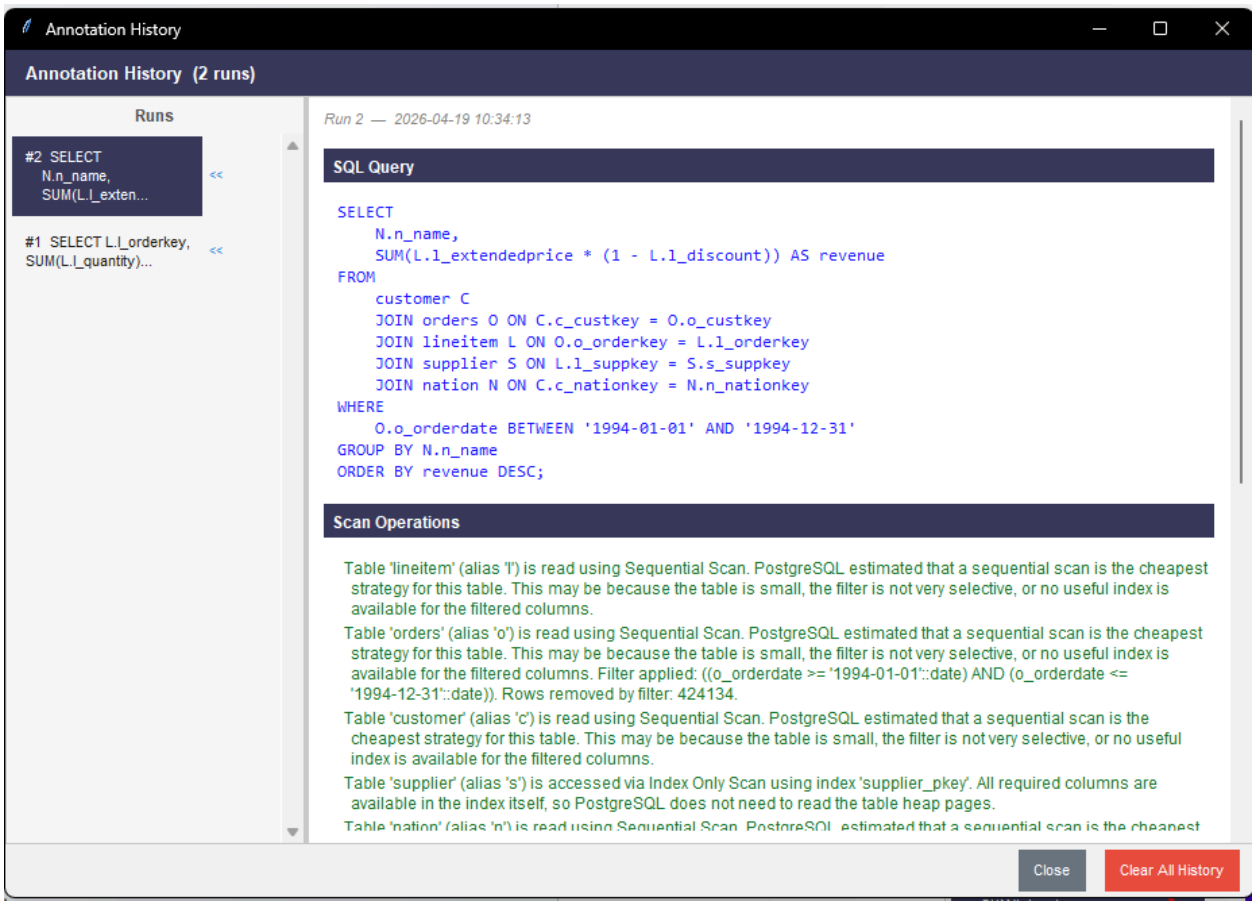


Figure 14. History Window

The annotation history is stored in-memory as a list of run records, each containing the timestamp, original SQL and the annotations produced. The History button opens a modal popup that is split into 2 sub-panels. On the left is a scrollable run list on the left showing a numbered preview of each query. On the right, it displays the full annotated output for the selected run.

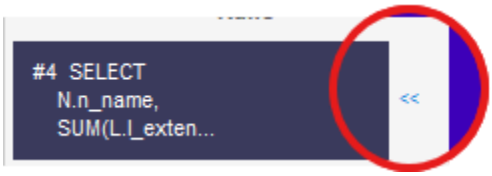


Figure 15. Restore button

Each run entry also provides a small restore button that loads the original SQL back into the left panel's query input.

4.7. Startup Checks

Before the GUI window is rendered, project.py performs a startup check.

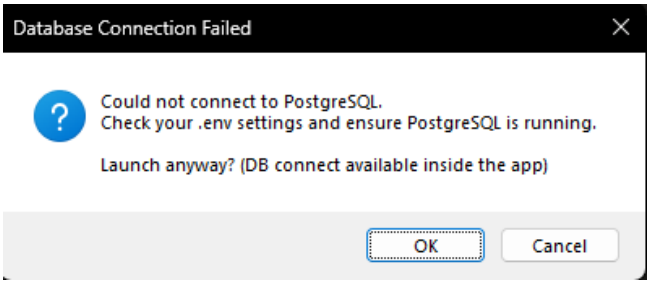


Figure 16. Messagebox on Startup Failure

It attempts to validate that PostgreSQL is reachable; if the connection fails at startup, a dialog notifies the user before the main window is opened. Though the user still can enter the program albeit the broken connection if they wish to once they resolve it instead of restarting the whole program.

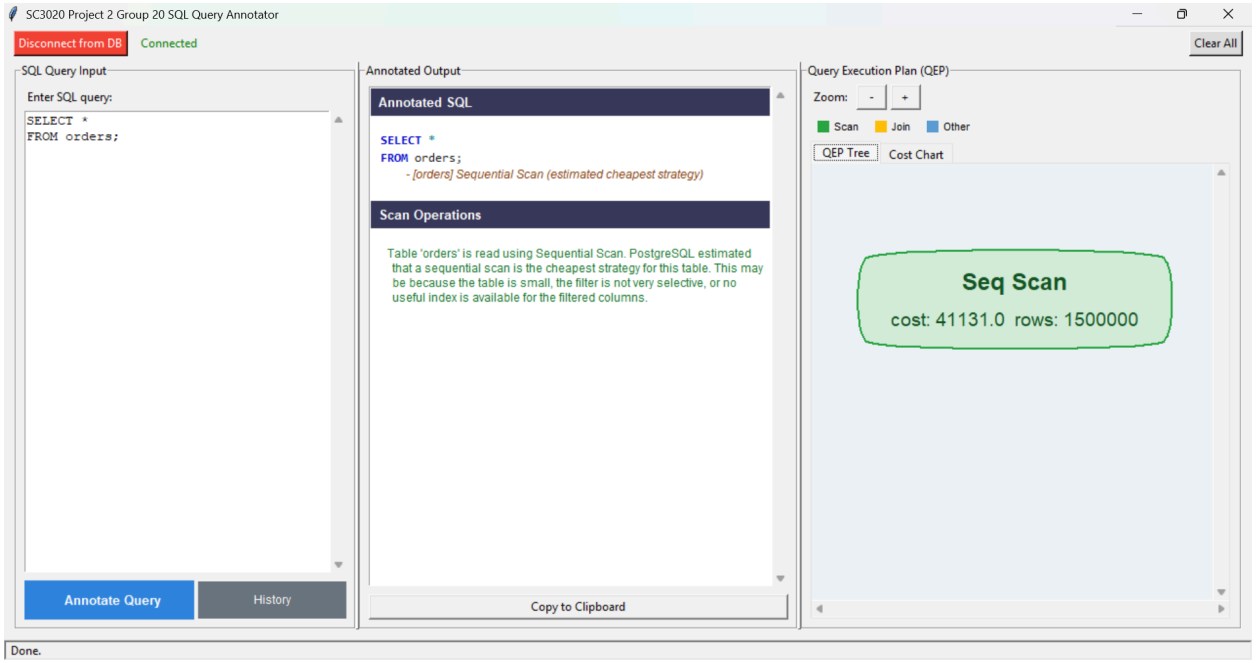
5. Query

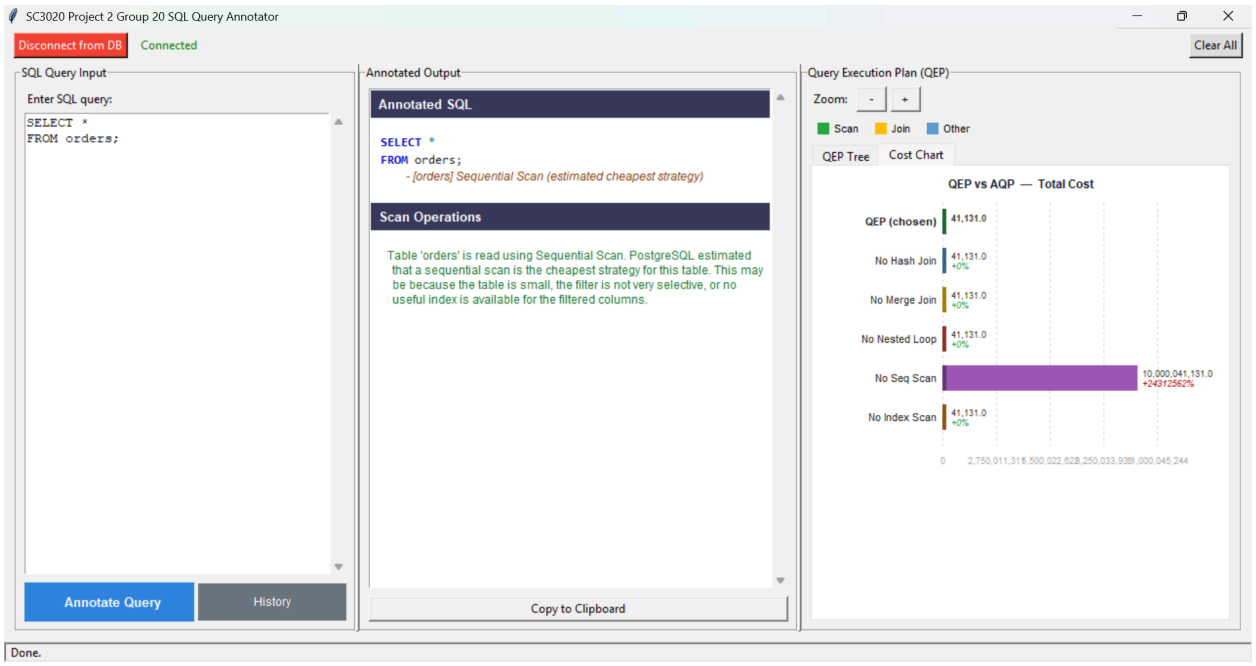
5.1 Sequential Scan

Query 1: Full Relation

```
SELECT *
FROM orders;
```

Results:



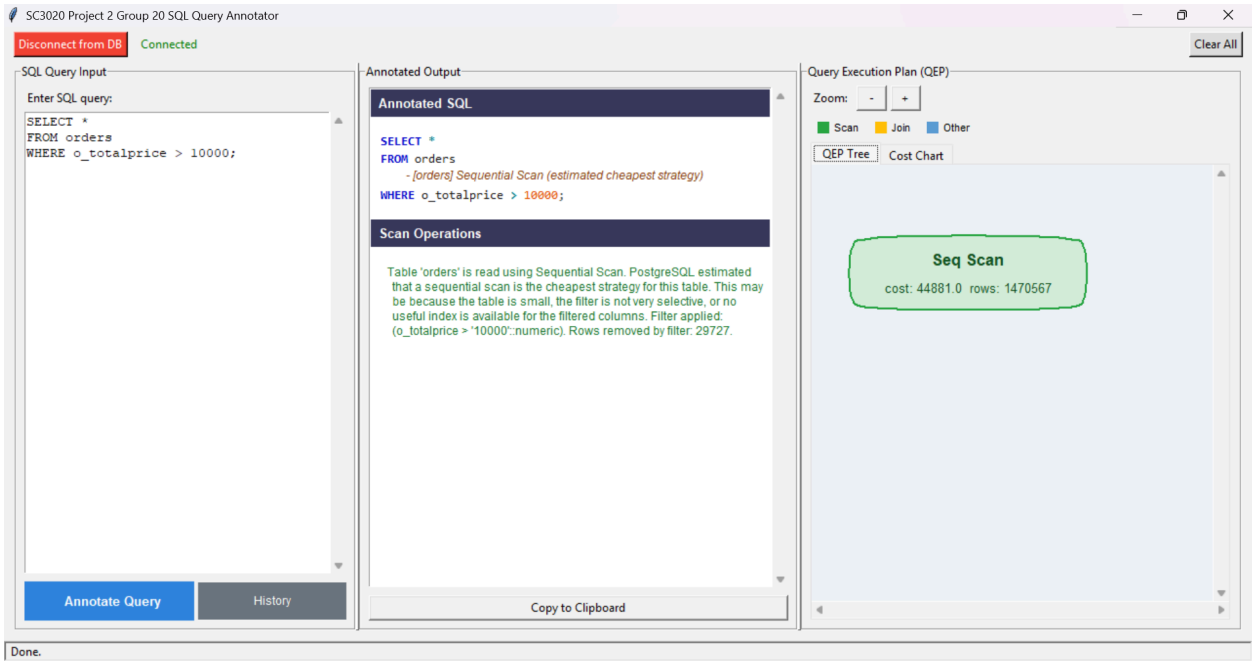


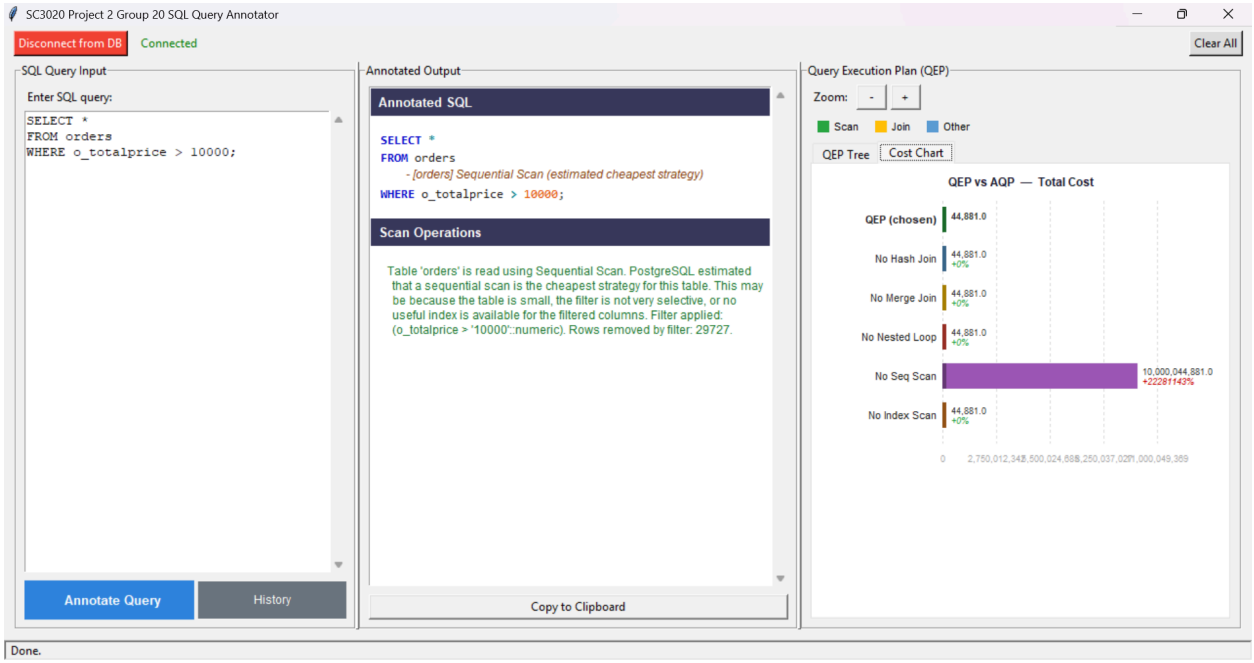
As we would expect, the most straightforward method to emit all tuples in a relation is done by scanning all the blocks of the relation sequentially, without any additional work.

Query 2: Single Relation with selection on non-indexed attribute

```
SELECT *
FROM orders
WHERE o_totalprice > 10000;
```

Results:



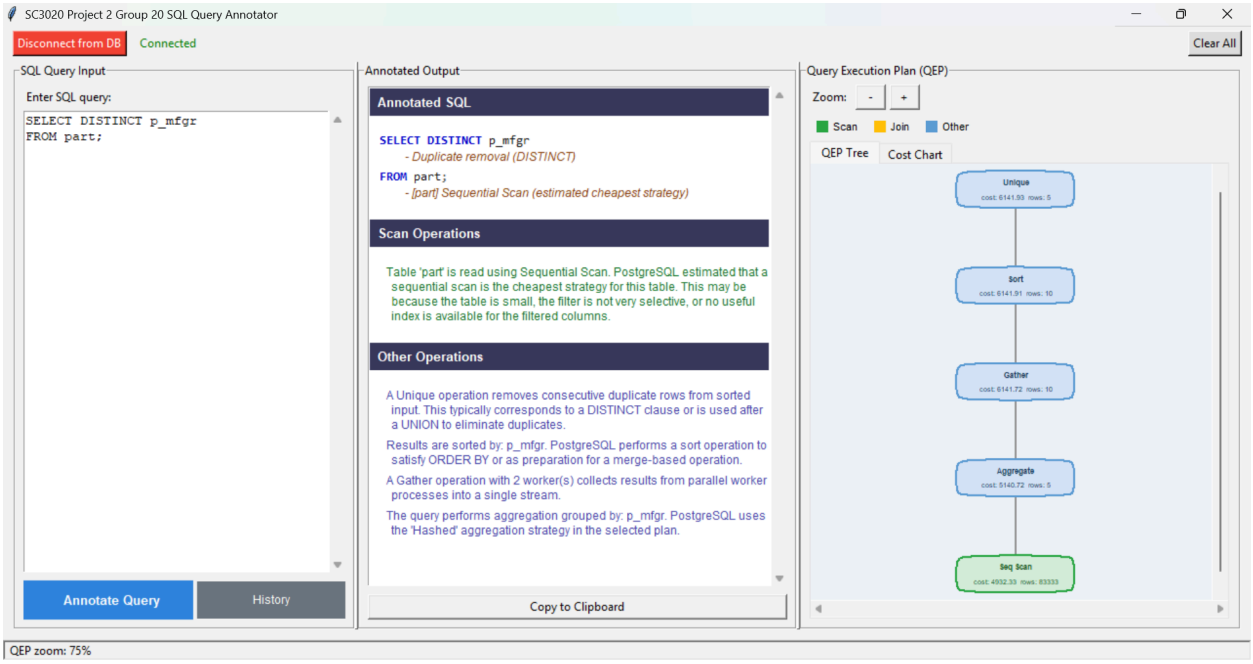


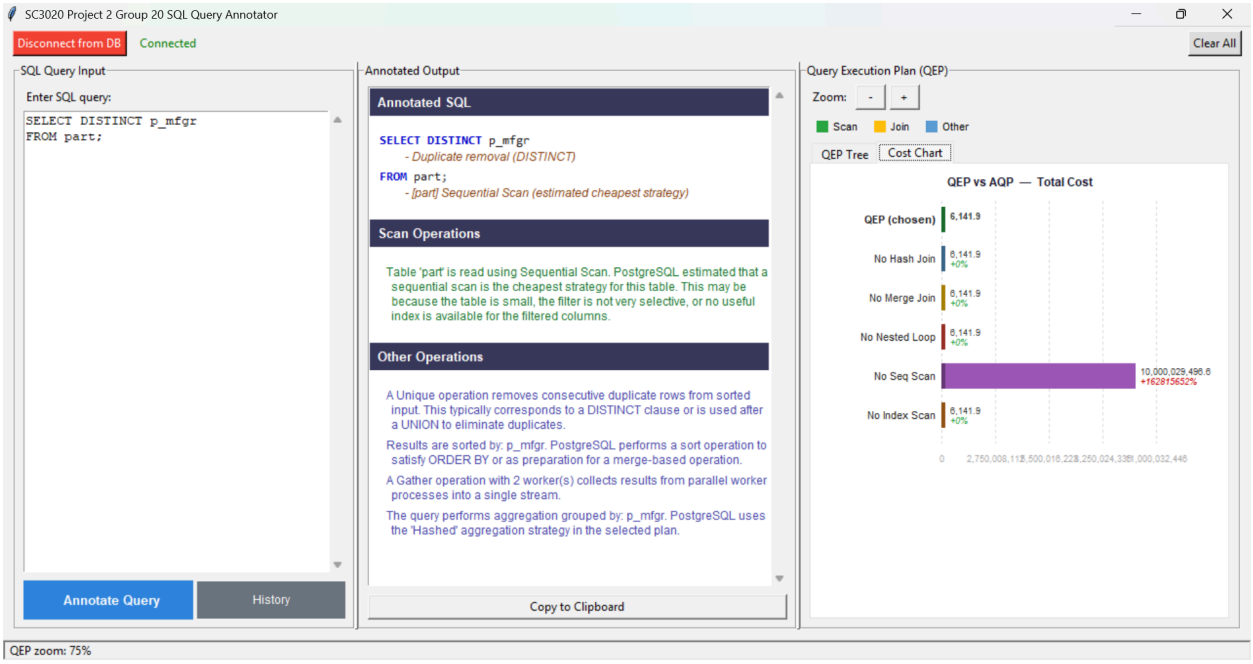
For a selection on an attribute that has no index, a sequential scan is still preferred as each tuple in the relation must be checked for the condition and there exist no other data structures to reduce the minimum tuples that must be checked. As such, the least possible work is still just a sequential scan.

Query 3: Duplicate Removal with DISTINCT

```
SELECT DISTINCT p_mfgr
FROM part;
```

Results:





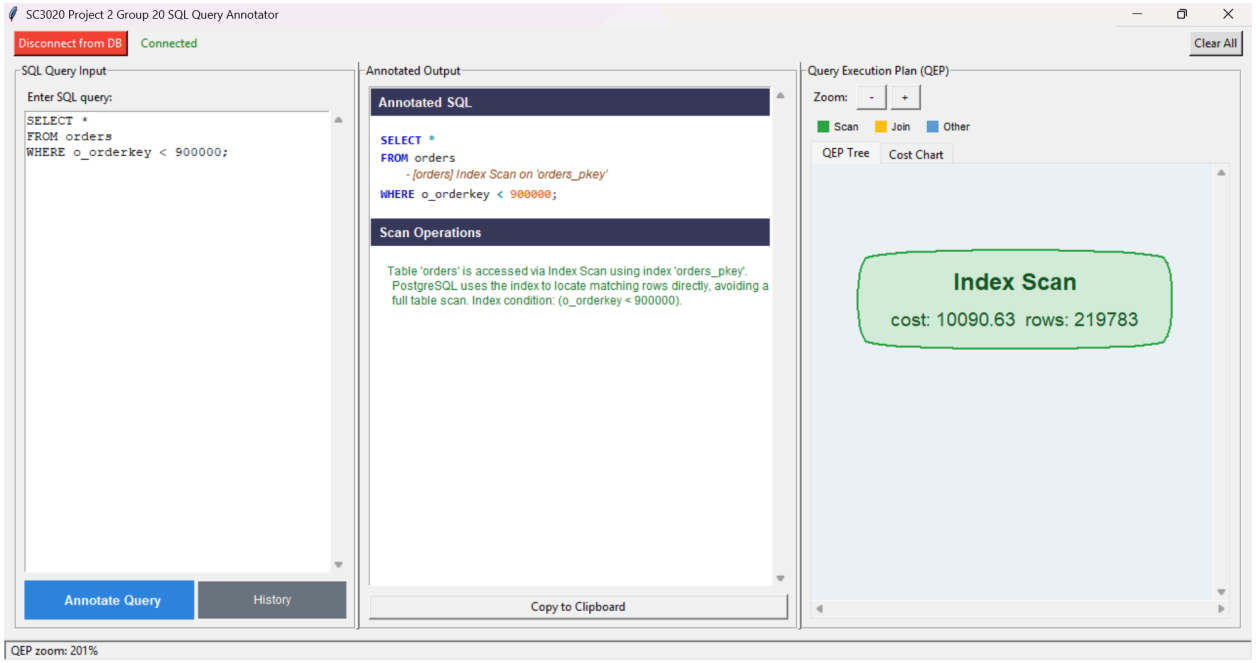
The steps when performing SELECT DISTINCT involve aggregation by grouping by the selected attribute(s), gathering to collect results executed in parallel, then sorting the results such that picking out duplicates is done more easily. It starts from a sequential scan again as the attribute is not indexed.

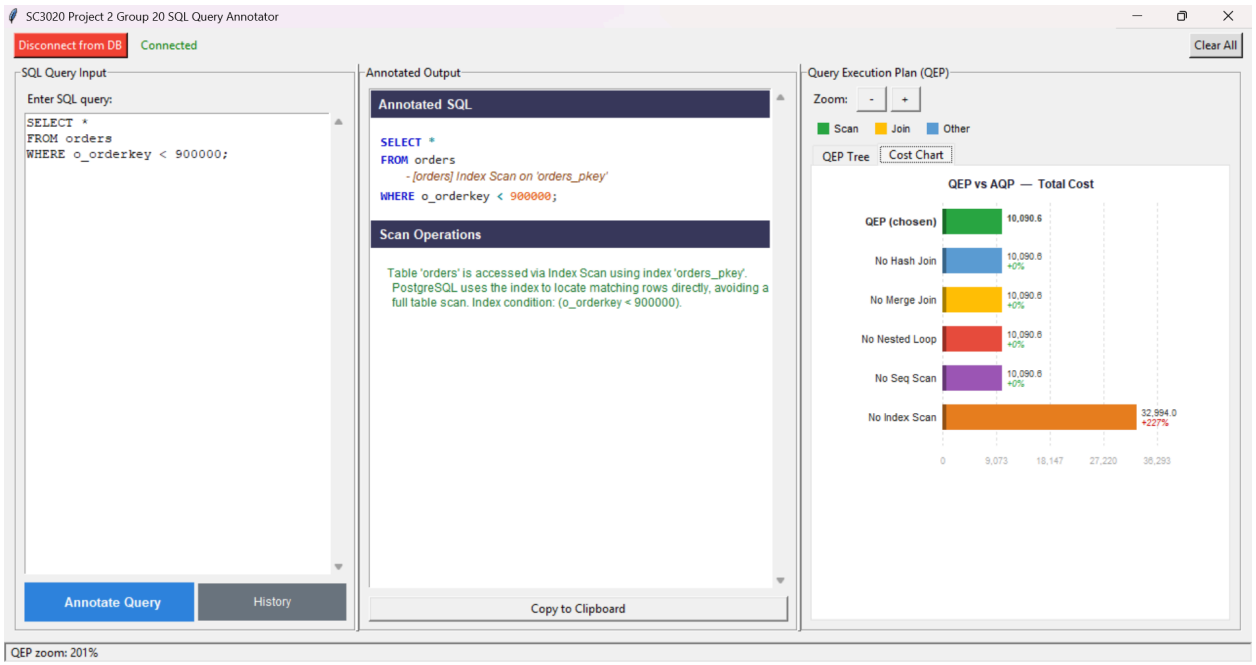
5.2 Index Scan

Query 1: Selection of full tuples fulfilling a condition on indexed attribute

```
SELECT *
FROM orders
WHERE o_orderkey < 900000;
```

Results:





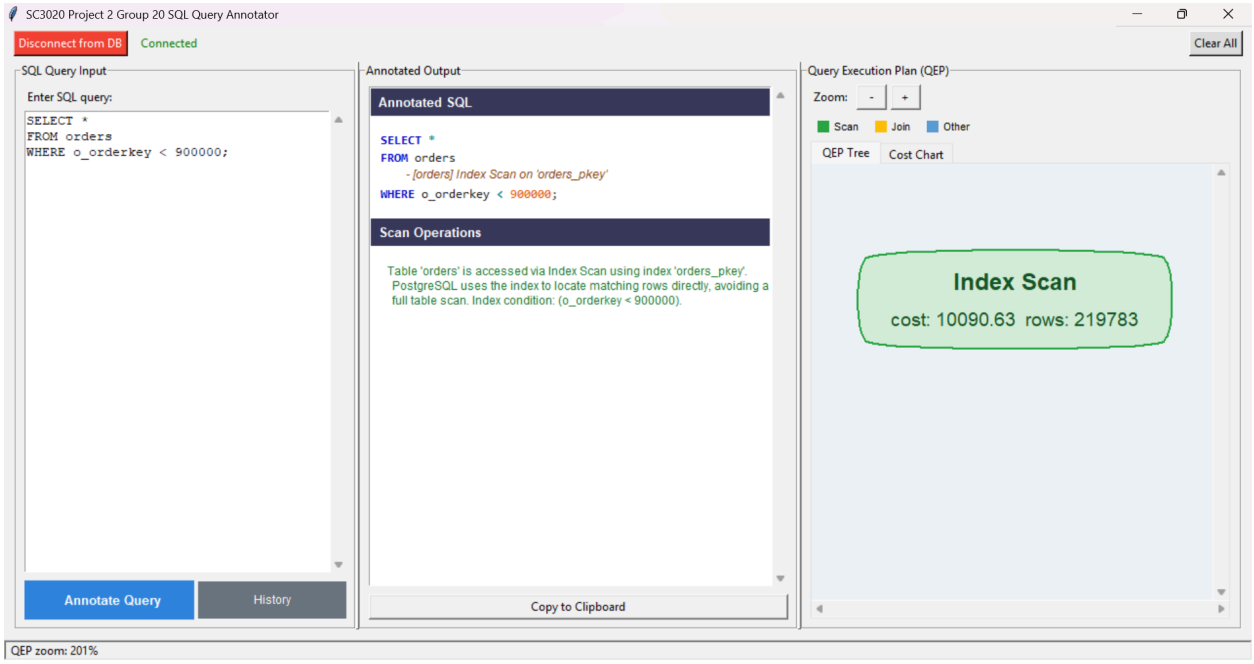
Now that the selection filter is based on a primary key, which is indexed, the index can be exploited to retrieve the tuples meeting the condition since the fulfilment of the select condition can already be checked within the index itself.

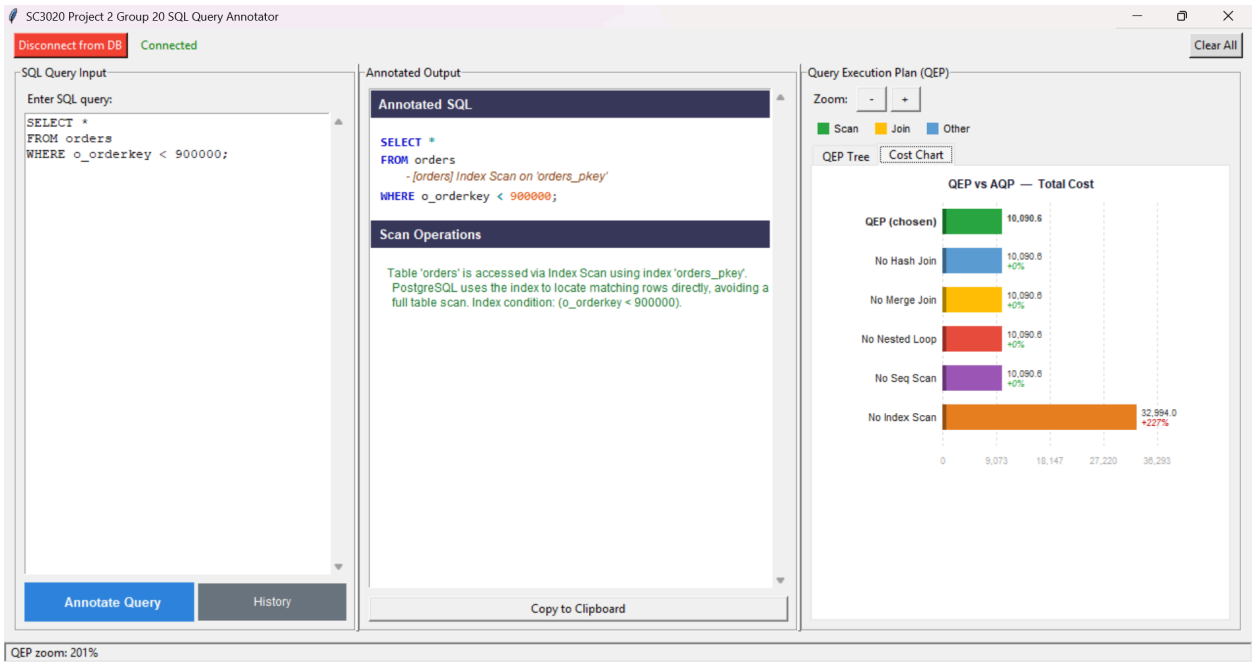
5.2.1 Index-Only Scan

Query 1: Selection of indexed attribute

```
SELECT o_orderkey
FROM orders
WHERE o_orderkey < 900000;
```

Results:





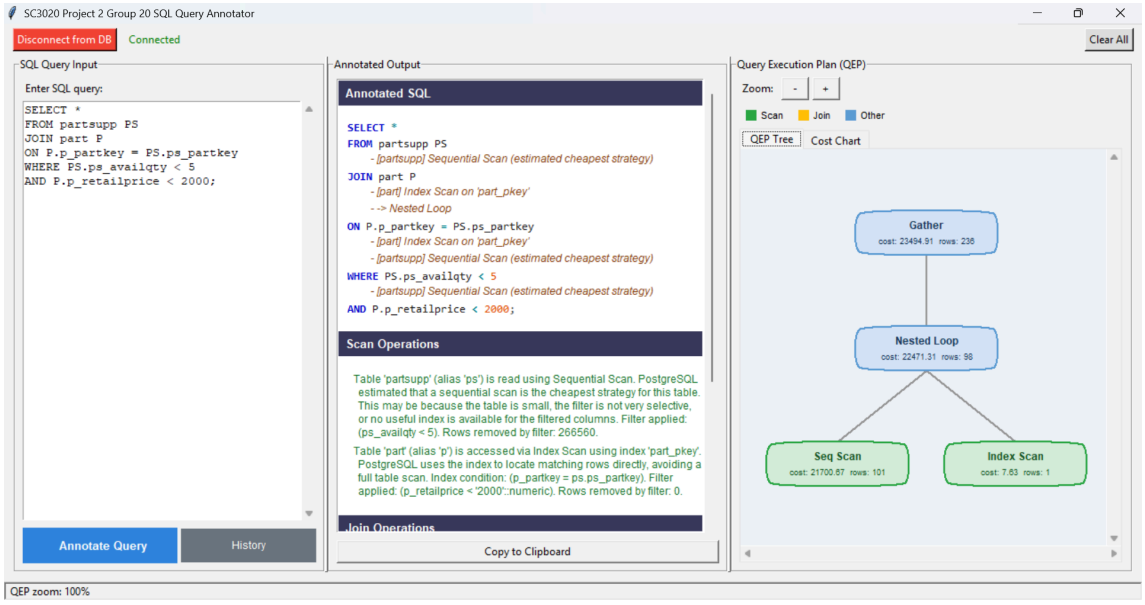
This time, as we only wish to project the primary key, the information can be found with a range query from [-∞,900000) on the B-Tree index, thus the index-only scan without need to access the tuples in disk blocks is used.

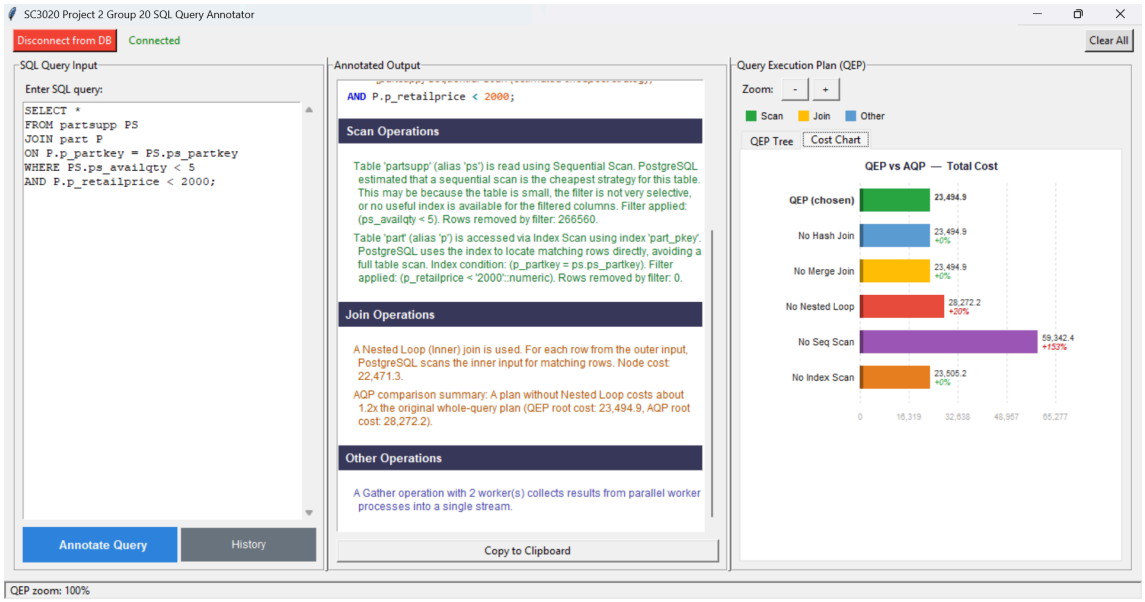
5.3 Nested Loop

Query 1: Join two small relations after selections on non-indexed attributes

```
SELECT * FROM partsupp PS
JOIN part P ON P.p_partkey = PS.ps_partkey
WHERE PS.ps_availqty < 5
AND P.p_retailprice < 2000;
```

Results:





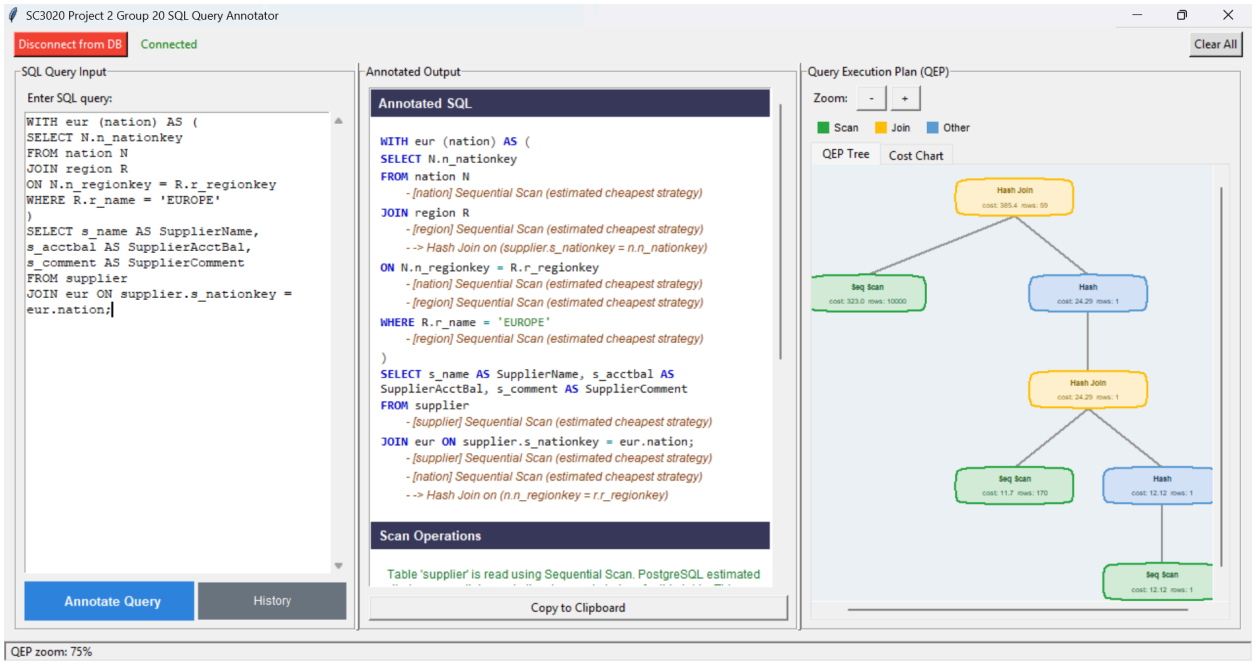
The selection conditions ensure that the resulting relations post-selection are relatively small, where Nested Loop joins perform decently. The sequential scan on relation partsupp allows the collection of the foreign key partkey, which is collated and used to access tuples in relation part, further saving time.

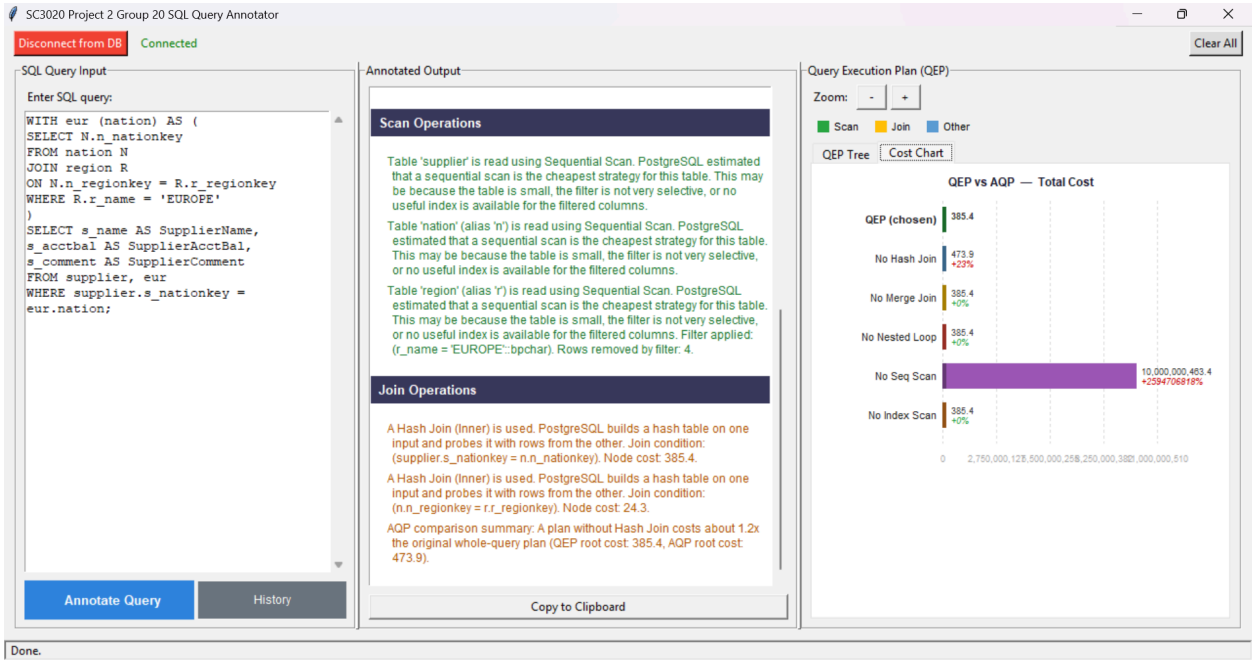
5.4 Hash Join

Query 1: A primary key-foreign key join nested in another

```
WITH eur (nation) AS (
SELECT N.n_nationkey
FROM nation N
JOIN region R
ON N.n_regionkey = R.r_regionkey
WHERE R.r_name = 'EUROPE'
)
SELECT s_name AS SupplierName, s_acctbal AS SupplierAcctBal, s_comment
AS SupplierComment
FROM supplier, eur
WHERE supplier.s_nationkey = eur.nation;
```

Results:





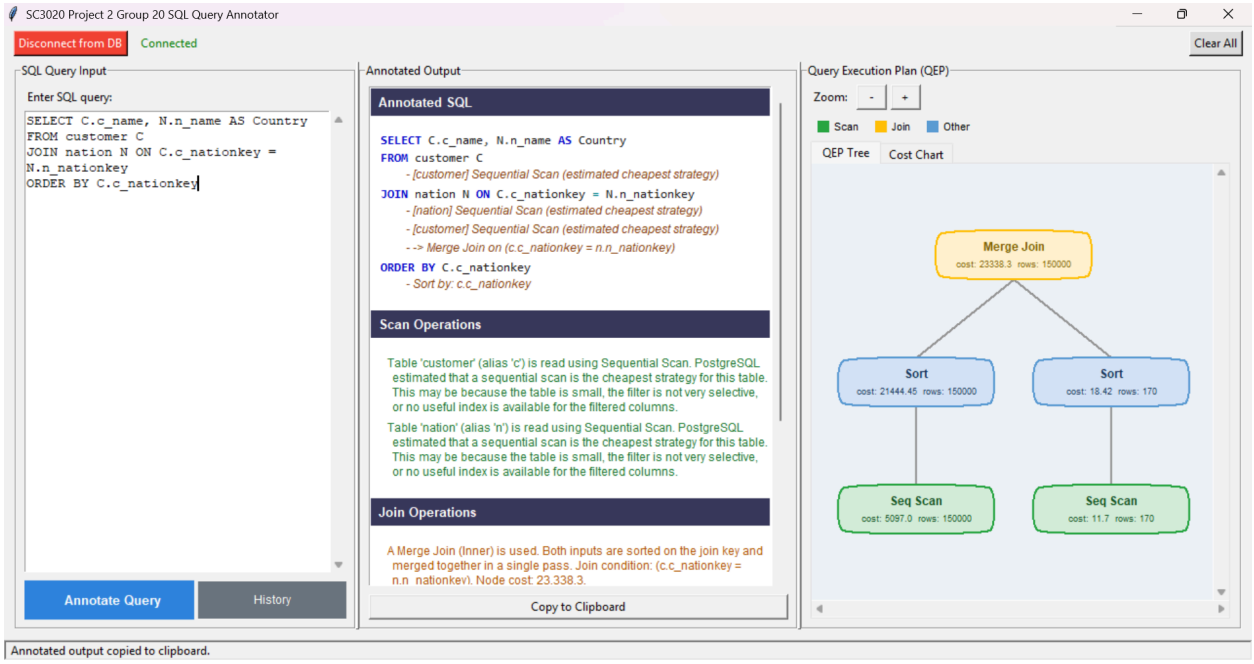
This query involves first finding the nations which are in region ‘EUROPE’, for which no indexes exist for attribute regionkey, hashing the values and performing a hash join, then repeating for the attribute nationkey found in the nation and supplier relations. This also shows that the application works with nested queries and WITH AS statements.

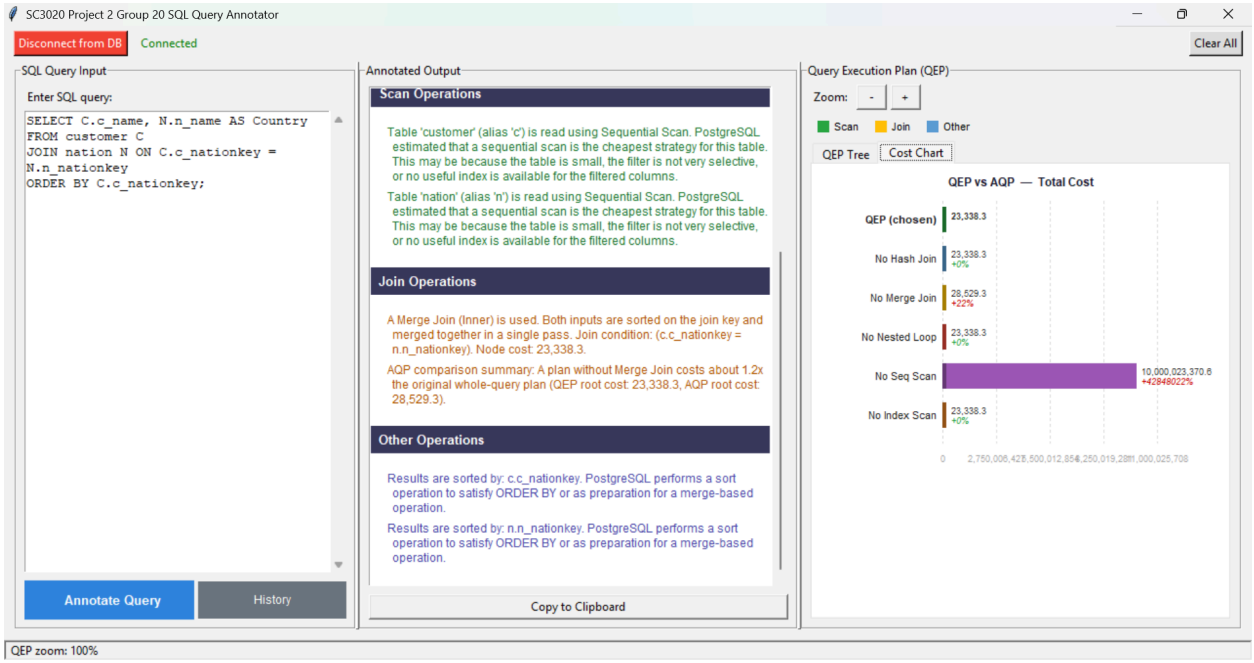
5.5 Merge Join

Query 1: Primary key Foreign key join with ORDER BY join attribute

```
SELECT C.c_name, N.n_name AS Country
FROM customer C
JOIN nation N ON C.c_nationkey = N.n_nationkey
ORDER BY C.c_nationkey;
```

Results:





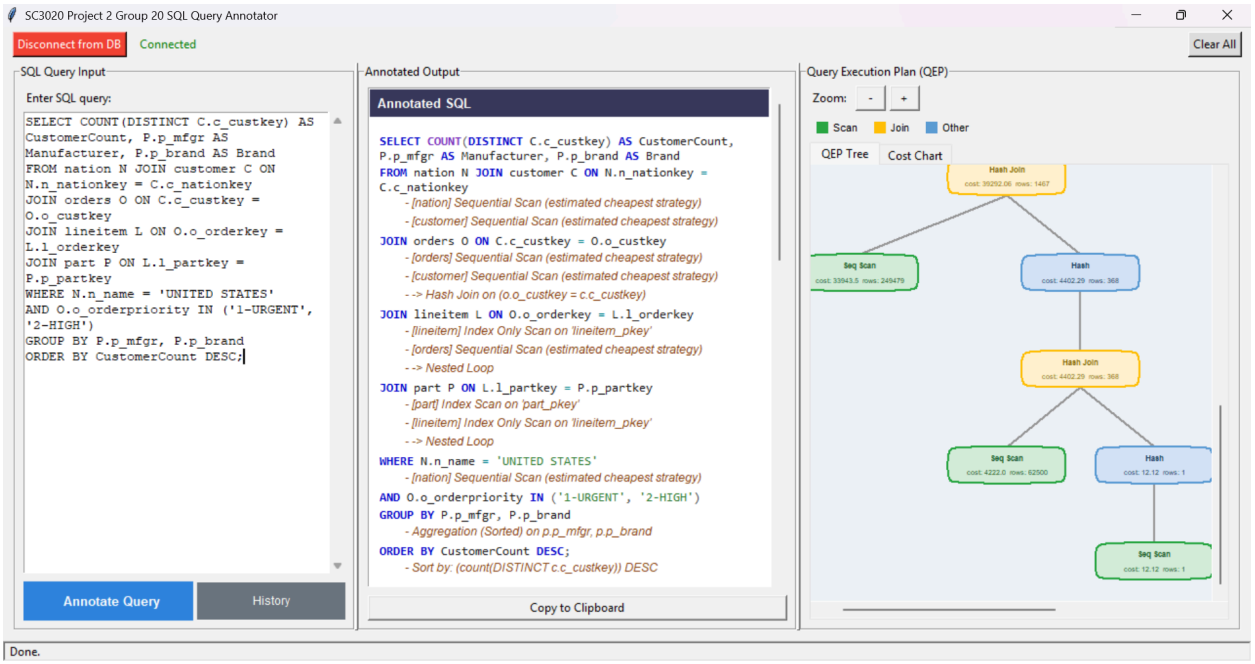
The ORDER BY clause ensures both relations should be sorted by attribute nationkey. This then makes it straightforward to perform a Sort-Merge Join since the resulting output will also be sorted.

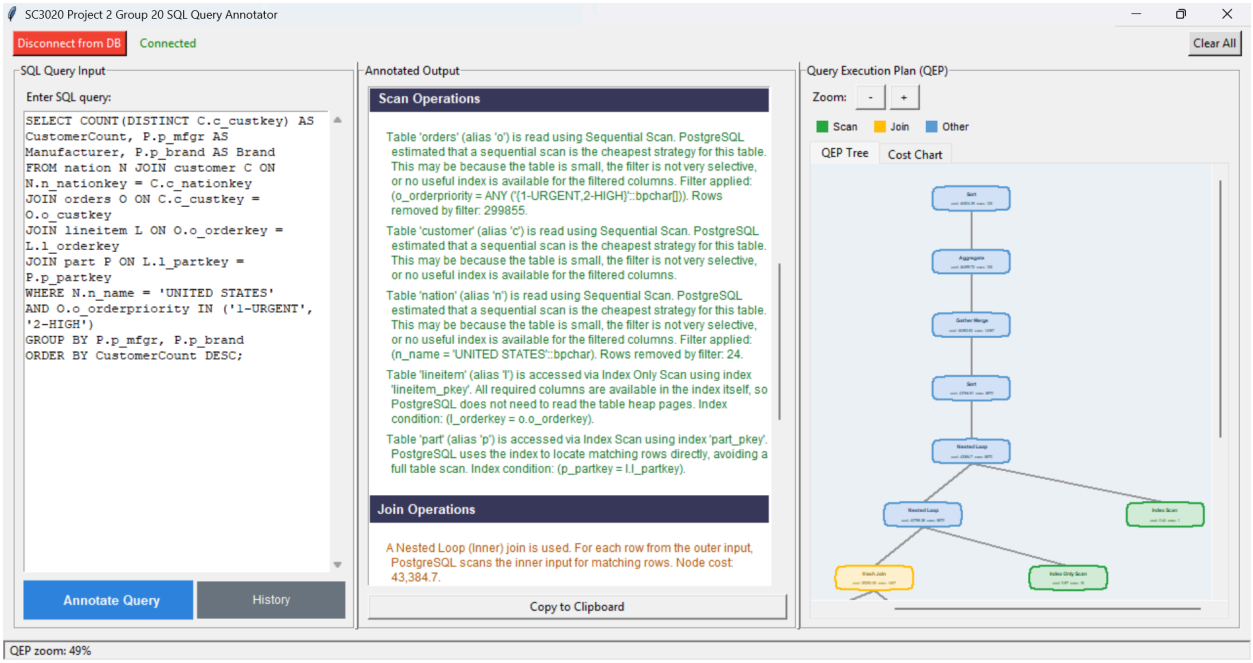
5.6 More Complex Query Examples

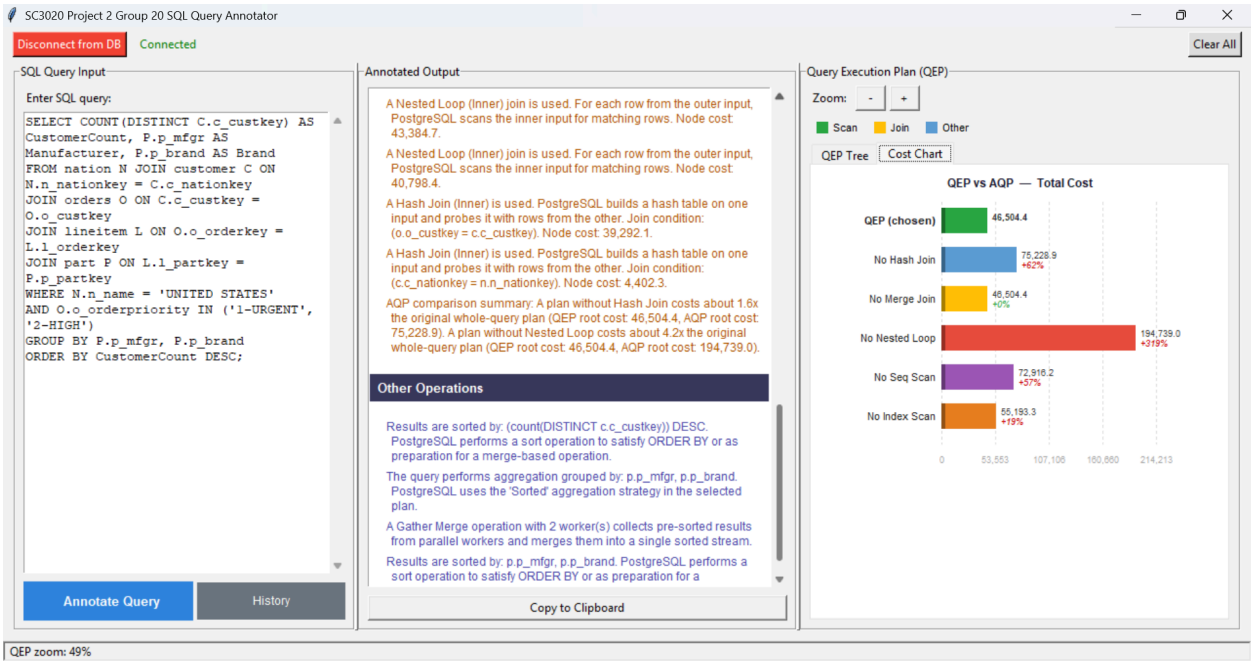
Query 1: Finding the number of customers that order from various manufacturer-brand combinations

```
SELECT COUNT(C.c_name) AS CustomerCount, P.p_mfgr AS Manufacturer,
P.p_brand AS Brand
FROM nation N JOIN customer C ON N.n_nationkey = C.c_nationkey
JOIN orders O ON C.c_custkey = O.o_custkey
JOIN lineitem L ON O.o_orderkey = L.l_orderkey
JOIN part P ON L.l_partkey = P.p_partkey
WHERE N.n_name = 'UNITED STATES'
AND O.o_orderpriority IN ('1-URGENT', '2-HIGH')
GROUP BY P.p_mfgr, P.p_brand
ORDER BY CustomerCount DESC;
```

Results:



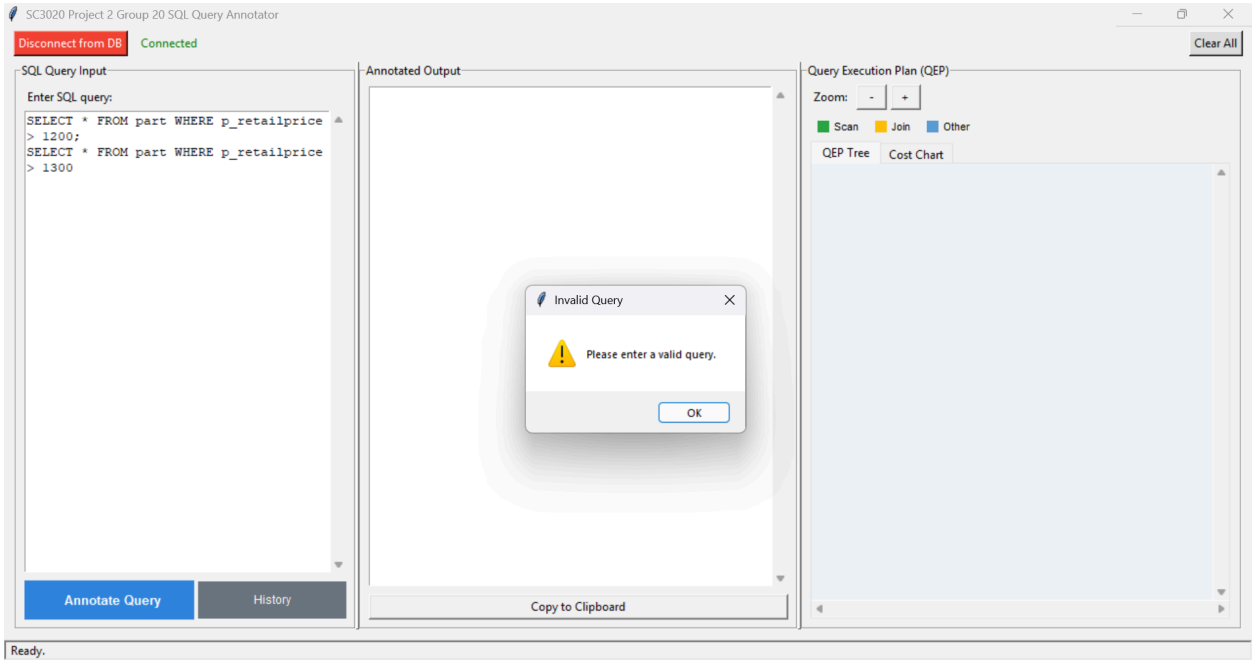




Nested Loop may be favoured due to a large number of buffers available. As it is used with the later joins, it has a larger impact since the number of tuples is likely to be larger. Merge join is the only option not chosen here because there is no opportunity to utilise a Sort-Merge join. Since the attribute that the output is to be sorted by is an aggregate, the sorting cannot be done beforehand.

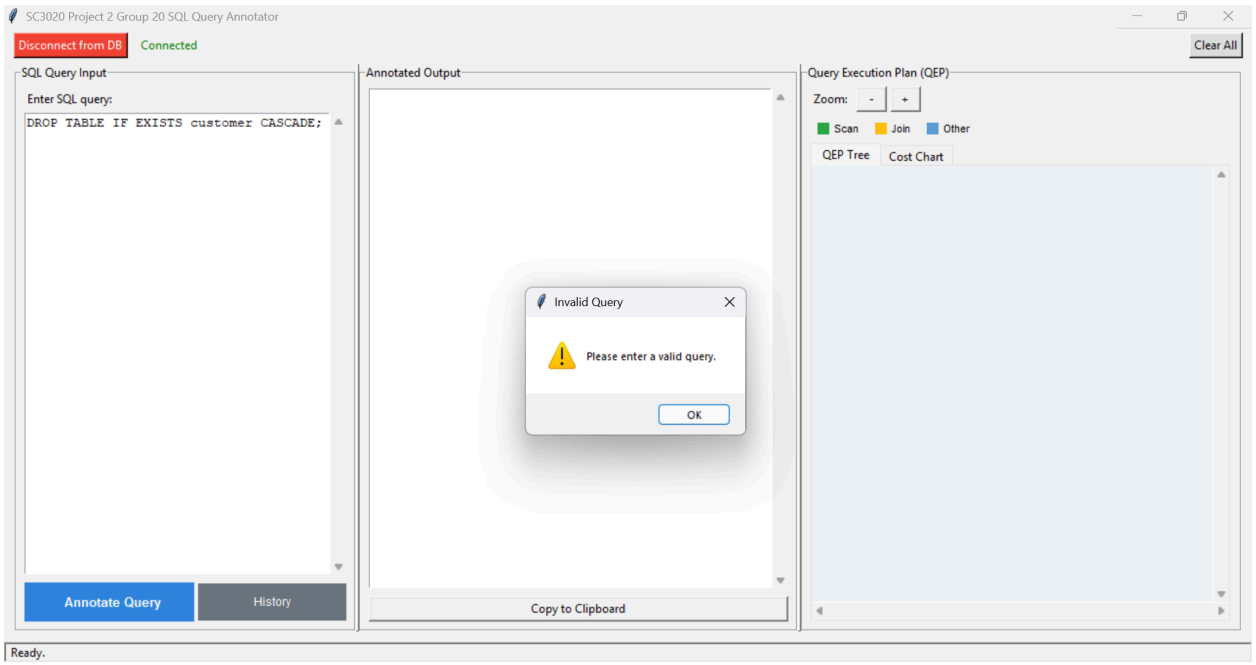
5.7 Handled Exceptions

Multiple Queries



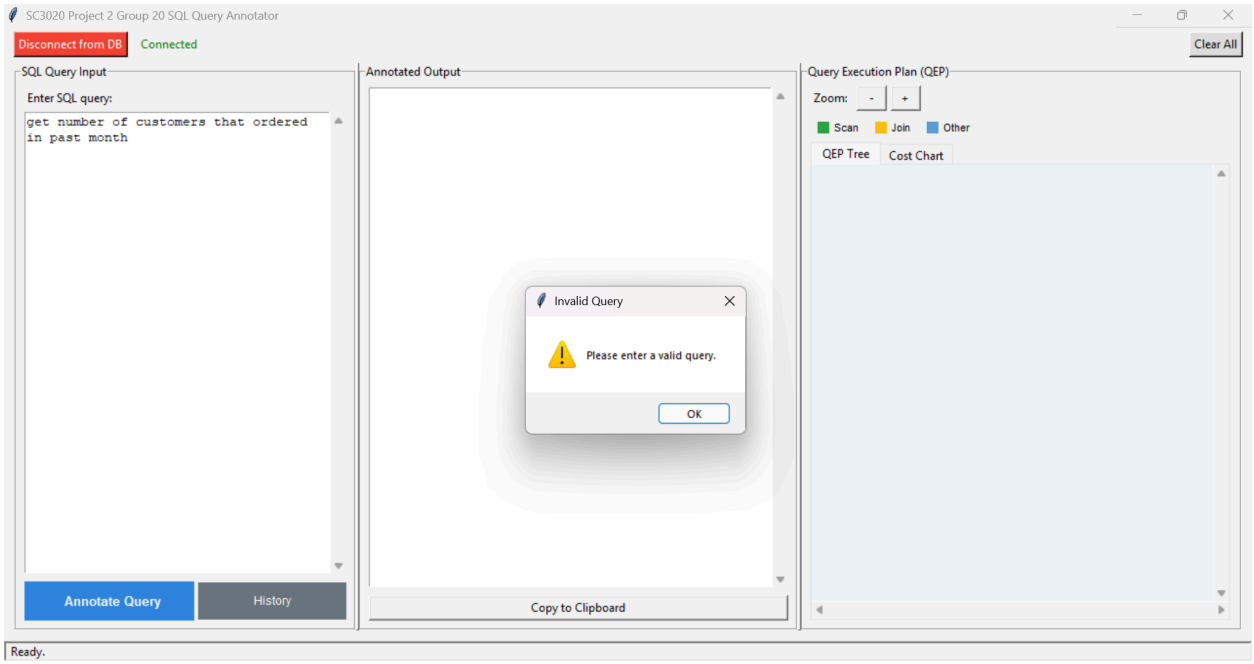
Semicolons mid-query are identified and flagged as invalid.

Attempts to Change Data in database



Dangerous keywords involving making any changes to the database, e.g. DROP, CREATE, UPDATE, are flagged as invalid.

Non-Queries



If the preprocessor cannot identify something in the structure of an SQL query, it will flag it as invalid.

6. Limitations

Limitation	Description
SQL Query Restrictions	The program only supports SELECT statements as the other queries are out of scope. Multi-statement queries are also rejected as string matching for a gap is out of scope. Lack of auto-correct for query as that would require some natural language processing which is out of scope
Infrastructure & Persistence	Since the program is ran locally, the History Logs are only stored in memory and the run records are lost when the application closes,
Lack of SQL interface	The program is unable to display the actual results retrieved from the database as the execution would be out of scope as it would not belong to any of the 4 files.